HDOC DAY–57 ACTIVITY

Maximum Subarray Sum Using Kadane's Algorithm in C

Name:	Aryan Pathak	Batch No.:	16
Course:	B.Tech CSE	Semester:	1

1. Aim

To write a C program that reads an integer array, passes the array to a function named **max_subarray_sum()** using pointers, and finds the maximum sum of a non-empty contiguous subarray using Kadane's algorithm.

2. Algorithm

1. Read the size n of the array.
2. Read n integer elements into the array.
3. Pass the address of the first element and the array size to max_subarray_sum().
4. Initialize current_sum and max_sum with the first array element.
5. For each remaining element, choose the larger of the element itself and current_sum + element.
6. Update max_sum whenever current_sum becomes greater than max_sum.
7. Print max_sum. Starting with the first element handles all-negative arrays and ensures the selected subarray is non-empty.

3. C Program

```
#include <stdio.h>

void max_subarray_sum(int *arr, int n)
{
    int current_sum = arr[0];
    int max_sum = arr[0];

    for (int i = 1; i < n; i++)
    {
        if (current_sum + arr[i] > arr[i])
            current_sum = current_sum + arr[i];
        else
            current_sum = arr[i];

        if (current_sum > max_sum)
            max_sum = current_sum;
    }

    printf("Maximum subarray sum: %d\n", max_sum);
}

int main(void)
{
    int n;

    printf("Enter the size of the array: ");
    if (scanf("%d", &n) != 1 || n <= 0)
    {
        printf("Invalid array size. Enter a positive integer.\n");
        return 1;
    }

    int arr[n];

    printf("Enter %d integer elements: ", n);
    for (int i = 0; i < n; i++)
```

```
    {
        if (scanf("%d", &arr[i]) != 1)
        {
            printf("Invalid input.\n");
            return 1;
        }
    }

    max_subarray_sum(arr, n);  /* Passes the array address */
    return 0;
}
```

4. Sample Test Cases

Test	Input array	Expected output	Explanation
1	-2 1 -3 4 -1 2 1 -5 4	Maximum subarray sum: 6	4 + (-1) + 2 + 1 = 6
2	-8 -3 -6 -2 -5 -4	Maximum subarray sum: -2	All values are negative; -2 is the largest.
3	5 -2 3 4	Maximum subarray sum: 10	The full array sums to 10.
4	0 0 0	Maximum subarray sum: 0	A non-empty subarray of zeroes has sum 0.
5	-5	Maximum subarray sum: -5	A one-element array is handled correctly.

5. Working of Kadane’s Algorithm

At each position, the algorithm decides whether to extend the previous subarray or start a new subarray from the current element. It keeps the best sum found so far. Since both sums start at arr[0] instead of zero, negative-only input is handled correctly.

6. Complexity Analysis

Time Complexity: O(n), because each element is processed once.
Auxiliary Space Complexity: O(1), because only a fixed number of extra variables are used (not counting the input array).

7. Result

The program passes the array to max_subarray_sum() using a pointer and calculates the maximum sum of a non-empty contiguous subarray using Kadane’s algorithm, including when all elements are negative.